

RnD Project: Utkarsh and Antriksh

22B0914, 22B1031

Contents

1	Final Outcome: System Protocol	3
1.1	Overview	3
1.2	Connection Phase	3
1.3	Heartbeat Mechanism	3
1.4	App Response to Server Instructions	3
1.4.1	(1) NAT Punch-Through Preparation	3
1.4.2	(2) Outgoing Ping Session	3
1.5	Data Upload	4
1.6	Upload Robustness	4
1.7	Reconnection Logic	4
2	Plots	6
3	Server-Side Implementation	9
3.1	ping_server.py - REST API Server	9
4	Android Client Implementation	9
4.1	MainActivity.kt - User Interface and Location Services	9
4.2	PingService.kt - Background Network Testing	10
5	RestApiClient.kt - HTTP Communication Layer	10
6	Server-Controlled Ping Instructions	10
6.1	Heartbeat Response Structure	10
6.2	Technical Implementation	10
7	Database Design	11
7.1	Clients Table	11
7.2	Ping Sessions Table	11
7.3	Ping Results Table	12
7.4	Heartbeats Table	12
8	JSON Data Structures	12
8.1	Registration Request Structure:	12
8.2	Complete Session Upload	12

9	Workflows	14
9.1	Initialization	15
9.2	Server Connection	16
9.3	Hearbeats	18
9.4	Pings	20
10	Summary	24
11	System Architecture	24
11.1	Android Client (The "Probe")	24
11.2	Backend Infrastructure (The "Server")	25
12	Core Functionality & Protocol Flow	25
12.1	Connection and Registration	25
12.2	The Heartbeat and Instruction Loop	25
12.3	Ping Execution (Two Types Simultaneously)	26
13	Data and Protocols Summary	26
14	Key File Analysis	27

1 Final Outcome: System Protocol

1.1 Overview

The end-to-end protocol describes how the server, the mobile application, and the background sender component interact to perform coordinated heartbeat, NAT traversal, and dual-direction ping sessions for measurement. This section details the complete workflow.

1.2 Connection Phase

1. The server is running (currently hosted on the Dragon server).
2. The user opens the mobile application and presses the **Connect** button.
3. The application sends a **connect** API request to the server.
4. The server generates a unique identifier for the device of the form:

`device_model_{random_string}`

It then sends a **connect success** response back to the application.

5. The app now enters **Connected Mode**.

1.3 Heartbeat Mechanism

- The application sends a heartbeat every T minutes (currently $T = 60$).
- The server receives the heartbeat and responds with an **instruction packet** containing:
 - The protocol to use for the ping session (UDP, ICMP, or TCP),
 - The target domain/server to ping,
 - The delay before the ping session begins (currently exponentially distributed).

1.4 App Response to Server Instructions

Upon receiving instructions, the app performs two parallel actions:

1.4.1 (1) NAT Punch-Through Preparation

1. A new thread is created.
2. The thread waits for the specified delay.
3. It then:
 - Opens a UDP port and begins listening.
 - Sends a UDP “ready” packet to the *sender* component. This prepares NAT traversal and signals that the app is ready to receive UDP packets.
4. The sender(hosted on dragon server for creating server to app traffic) , running in the background, receives the ready signal and begins sending UDP packets to the app periodically, following the same periodic delay and total transmission duration specified in the instruction.

1.4.2 (2) Outgoing Ping Session

After waiting for the server-defined delay, the app initiates the outgoing ping session:

- For UDP (example case):

- The app sends periodic UDP packets to the server's UDP listener (currently hosted on the Dragon server).
- The listener responds with ACKs for each received packet.

1.5 Data Upload

Once both ping sessions are complete (app→listener and sender→app), the app:

1. Compiles the session results,
2. Sends a `store` API request to the server,
3. The server stores the data and replies with a `store success`.

1.6 Upload Robustness

- If an upload attempt fails, the app saves the data locally.
- Before the next heartbeat, the app attempts to retransmit all previously failed uploads.

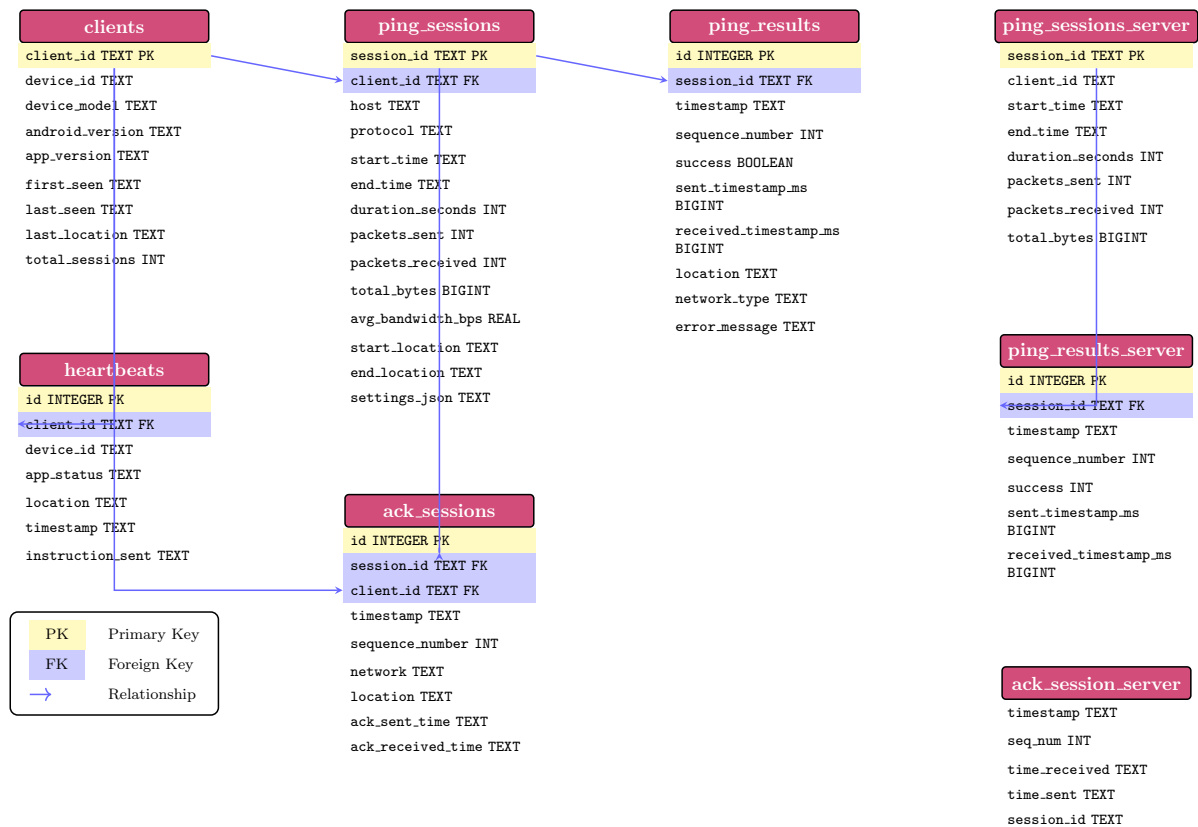


Figure 1: Complete Database Schema: Client and Server Tables

1.7 Reconnection Logic

If the app receives no response to a heartbeat:

- It enters **Reconnect Mode**.
- Instead of sending further heartbeats, it sends a `connect` request again.

- This is required because if the server restarts, it forgets past clients and only responds to registered (connected) clients.

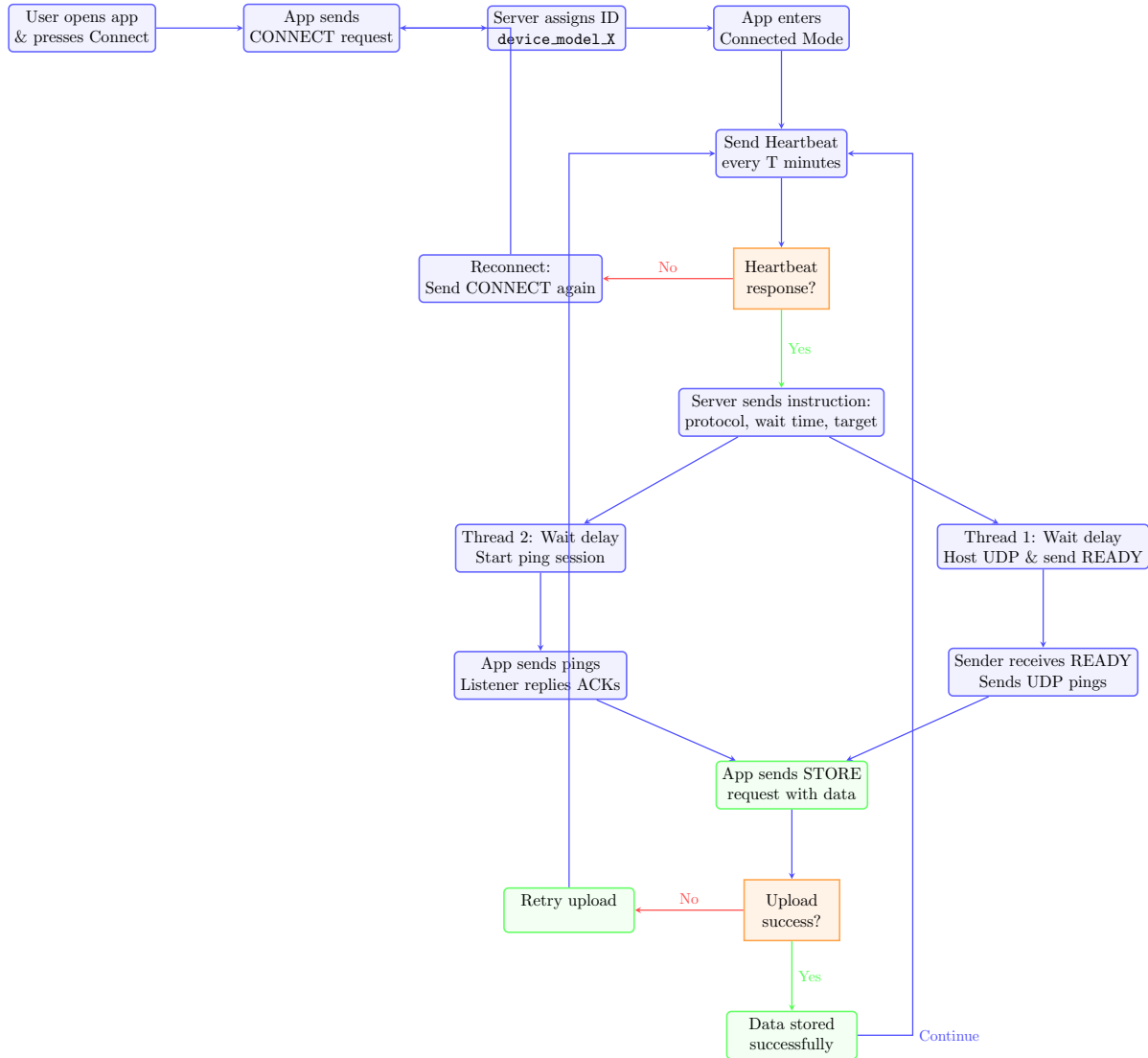


Figure 2: Overall Protocol Workflow

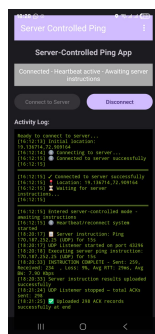


Figure 3: App User Interface

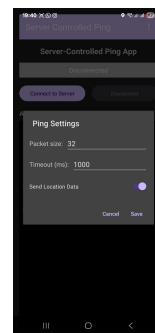


Figure 4: Location Settings

2 Plots

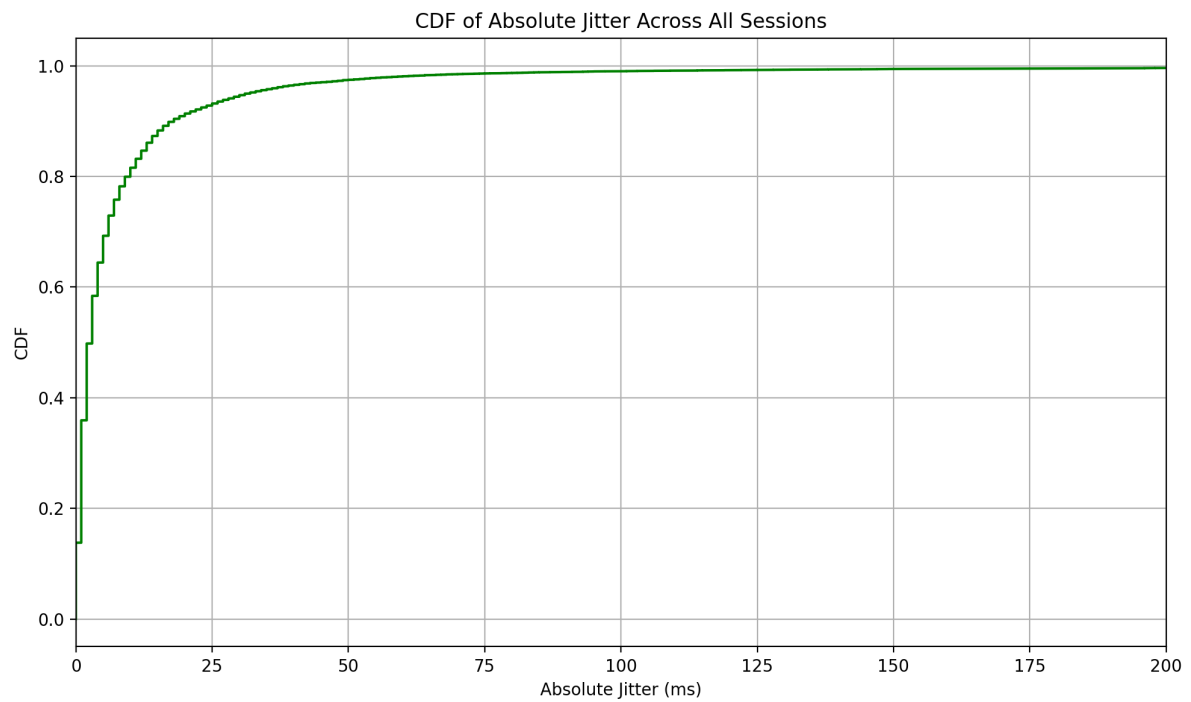


Figure 5: Absolute jitter CDF for 200 ms threshold

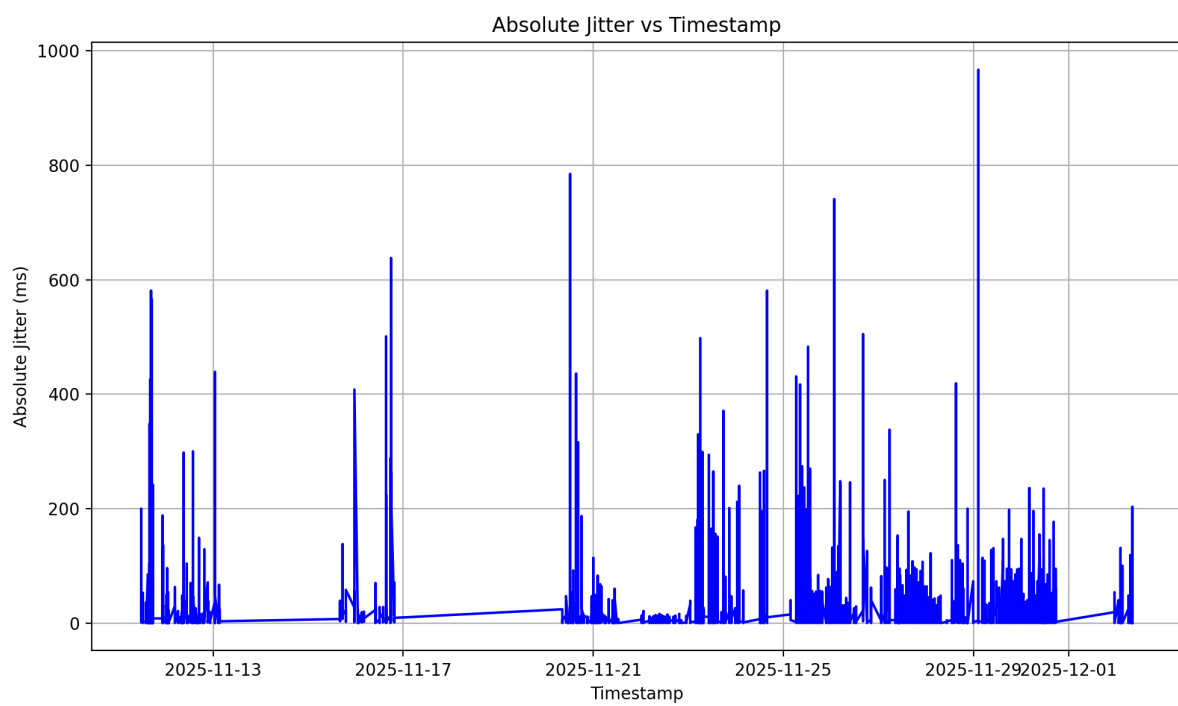


Figure 6: Absolute jitter vs Timestamp

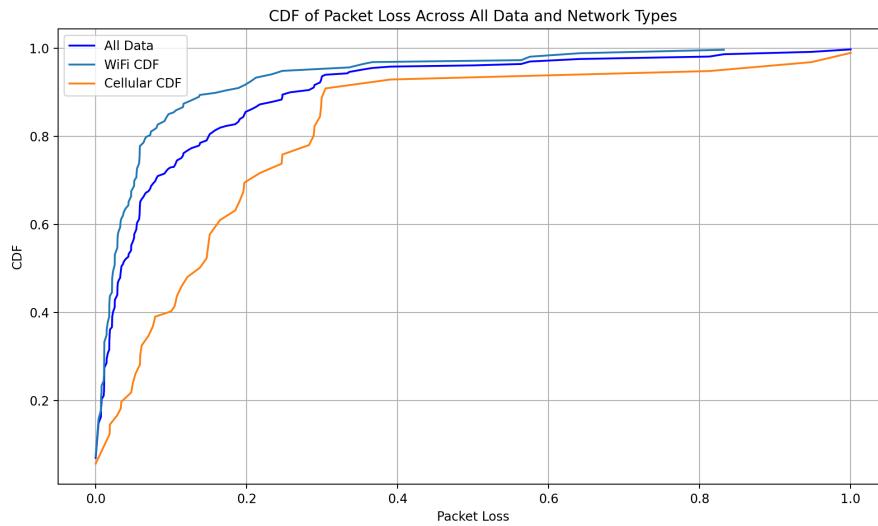


Figure 7: Packet Loss CDFs

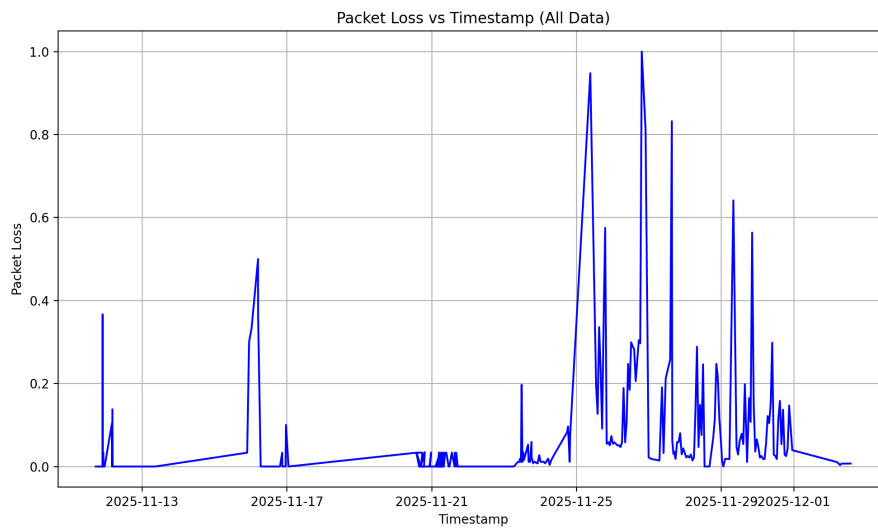


Figure 8: Packet Loss vs Timestamp (All Data)

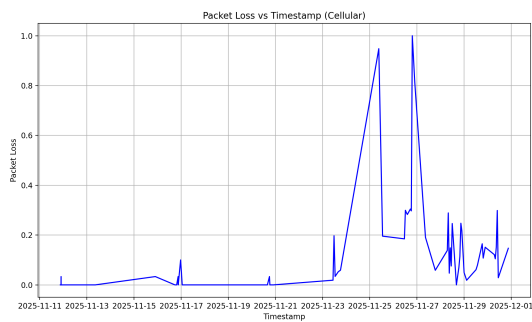


Figure 9: Packet Loss vs Timestamp (Cellular)

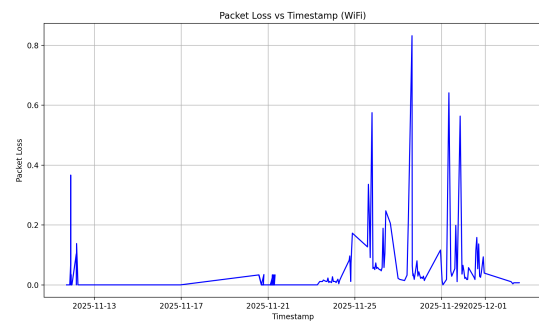


Figure 10: Packet Loss vs Timestamp (WiFi)

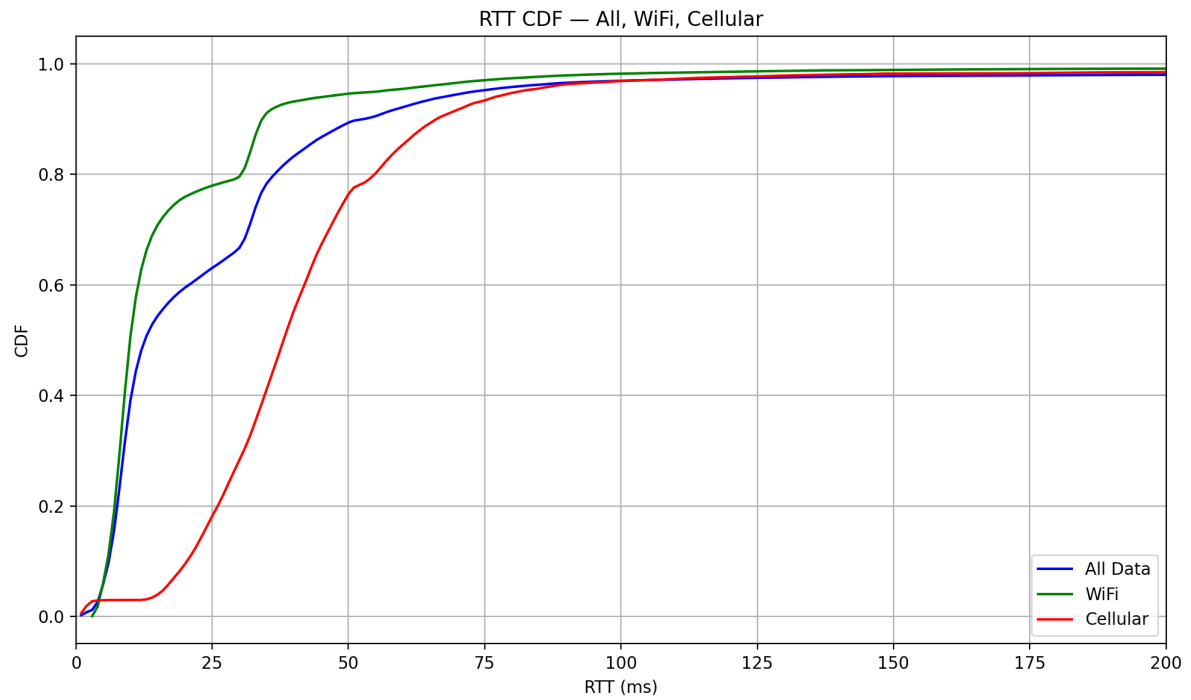


Figure 11: RTT CDF for All Data

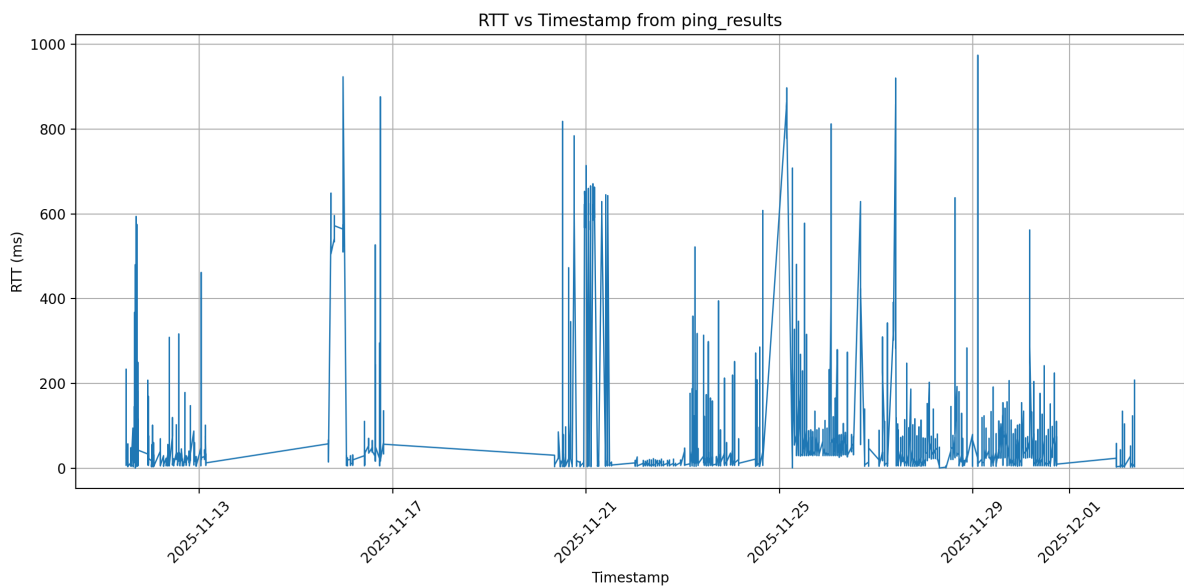


Figure 12: Packet Loss vs Timestamp (All Data)

3 Server-Side Implementation

August 21, 2025

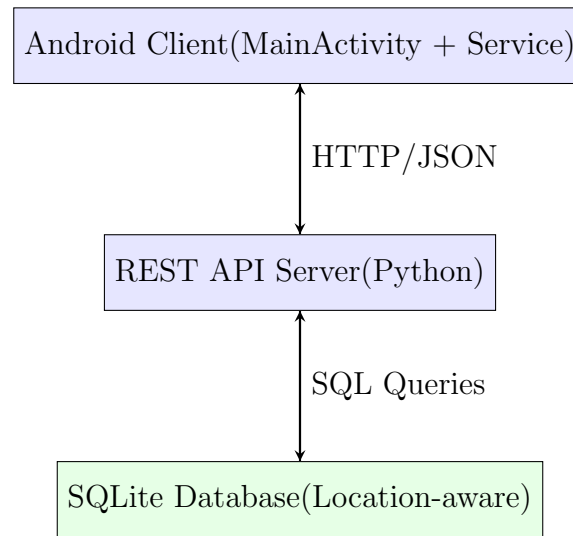


Figure 13: System Architecture Diagram

3.1 ping_server.py - REST API Server

The server forms the core data processing and storage engine of our system. We have so far implemented it using Python's built-in `HTTPServer` and `BaseHTTPRequestHandler` libraries, which handle RESTful API requests.

- **Database Management:** We created a custom `PingDataServer` class to manage SQLite operations. Thread safety is ensured using Python's `threading.Lock()`
- **API Handler:** The `PingRestApiHandler` class currently processes HTTP requests across nine endpoints

Endpoint	Method	Description
/api	GET	API information and version details
/api/status	GET	Server health check and status
/api/ping	GET	Simple connectivity test
/api/connect	POST	Device registration with location data
/api/heartbeat	POST	Periodic client status updates
/api/upload-session	POST	Complete ping session data upload
/api/clients	GET	Retrieve all registered clients
/api/clients/{id}	GET	Specific client information
/api/locations	GET	Location-based statistics

4 Android Client Implementation

4.1 MainActivity.kt - User Interface and Location Services

The main activity serves as the hub for user interaction, location tracking, and background ping operations. We have so far implemented Android's `LocationManager` and `LocationListener` to access GPS and network-based location providers.

- **UI Features:** The interface dynamically reflects service states and displays real-time logs with automatic scrolling. We have used service binding to ensure smooth communication between the UI and the background service.
- **Server Communication:** The device registers itself with the server, including location details, and maintains periodic heartbeat communication every five minutes.
- **Location Handling:** So far, location updates are requested every 10 seconds or after moving 10 meters (If permissions are enabled).

4.2 PingService.kt - Background Network Testing

The background service is implemented as a foreground service with persistent notifications to ensure continuous operation. It uses Kotlin coroutines with `Dispatchers.IO` for efficient, non-blocking network operations.

- **Ping Implementation:** We have implemented:
 - ICMP pings via system `ping` command
 - TCP pings through socket connections
 - UDP pings via `DatagramSocket`

Each ping session collects detailed statistics including response times, packet loss, and bandwidth estimates, with location stamps for geographic context.

- **Session Data Upload:** Upon session completion, data is uploaded to the server.

5 RestApiClient.kt - HTTP Communication Layer

The REST API client bridges the Android app and Python server using `HttpURLConnection` with timeout and error handling. It serializes and deserializes data in JSON format.

6 Server-Controlled Ping Instructions

The server implements a probabilistic response mechanism on each heartbeat request. With probability p , the server responds with ping instructions containing specific network testing parameters.

6.1 Heartbeat Response Structure

On each heartbeat, the server responds with a JSON object containing:

- **send_ping:** Boolean flag indicating whether to execute ping (probability p)
- **ping_host:** Target IP address or hostname for ping testing
- **ping_protocol:** Protocol specification (ICMP, TCP, or UDP)
- **ping_duration_seconds:** Duration of ping session in seconds
- **ping_interval_ms:** Interval between individual ping packets in milliseconds

6.2 Technical Implementation

```
# Heartbeat response with ping instructions
{
    "status": "success",
    "send_ping": true,
    "ping_host": "8.8.8.8",
```

```
"ping_protocol": "ICMP",  
"ping_duration_seconds": 60,  
"ping_interval_ms": 1000  
}
```

When `send_ping` is false, the client maintains standby mode and continues regular heartbeat communication without executing ping operations.

7 Database Design

The SQLite database implements four primary tables with location support:

7.1 Clients Table

```
CREATE TABLE clients (  
    client_id TEXT PRIMARY KEY,  
    device_id TEXT,  
    device_model TEXT,  
    android_version TEXT,  
    app_version TEXT,  
    first_seen TEXT,  
    last_seen TEXT,  
    last_location TEXT,  
    total_sessions INTEGER DEFAULT 0  
);
```

7.2 Ping Sessions Table

```
CREATE TABLE ping_sessions (  
    session_id TEXT PRIMARY KEY,  
    client_id TEXT,  
    host TEXT,  
    protocol TEXT,  
    start_time TEXT,  
    end_time TEXT,  
    duration_seconds INTEGER,  
    packets_sent INTEGER,  
    packets_received INTEGER,  
    packet_loss_percent REAL,  
    avg_rtt_ms REAL,  
    min_rtt_ms REAL,  
    max_rtt_ms REAL,  
    total_bytes BIGINT,  
    avg_bandwidth_bps REAL,  
    start_location TEXT,  
    end_location TEXT,  
    settings_json TEXT,  
    FOREIGN KEY (client_id) REFERENCES clients (client_id)  
);
```

7.3 Ping Results Table

```
CREATE TABLE IF NOT EXISTS ping_results (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  session_id TEXT,  
  timestamp TEXT,  
  sequence_number INTEGER,  
  success BOOLEAN,  
  rtt_ms REAL,  
  location TEXT,  
  error_message TEXT,  
  FOREIGN KEY (session_id) REFERENCES ping_sessions (session_id  
  )  
);
```

7.4 Heartbeats Table

8 JSON Data Structures

8.1 Registration Request Structure:

```
{  
  "device_id": "unique_android_id",  
  "app_version": "1.0.0",  
  "device_model": "Manufacturer Model",  
  "android_version": "API_LEVEL",  
  "location": "latitude,longitude",  
  "timestamp": milliseconds_epoch  
}
```

8.2 Complete Session Upload

```
{  
  "session_id": "session_timestamp_random",  
  "client_id": "device_identifier",  
  "host": "target_hostname_or_ip",  
  "protocol": "ICMP|TCP|UDP",  
  "start_time": "YYYY-MM-DD HH:MM:SS",  
  "end_time": "YYYY-MM-DD HH:MM:SS",  
  "duration_seconds": elapsed_time,  
  "packets_sent": total_packets,  
  "packets_received": successful_packets,  
  "packet_loss_percent": loss_percentage,  
  "avg_rtt_ms": average_response_time,  
  "min_rtt_ms": minimum_response_time,  
  "max_rtt_ms": maximum_response_time,  
  "total_bytes": bytes_transferred,  
  "avg_bandwidth_bps": calculated_bandwidth,  
  "start_location": "start_coordinates",  
}
```

```
"end_location": "end_coordinates",
"settings": {
  "packet_size": size_in_bytes,
  "timeout": timeout_milliseconds,
  "interval": ping_interval_ms,
  "tcp_port": tcp_port_number,
  "udp_port": udp_port_number
},
"ping_results": [
  {
    "timestamp": "result_timestamp",
    "sequence": sequence_number,
    "success": true_or_false,
    "rtt_ms": response_time,
    "location": "coordinates",
    "error_message": "error_if_any"
  }
]
```

29 August 2025

9 Workflows

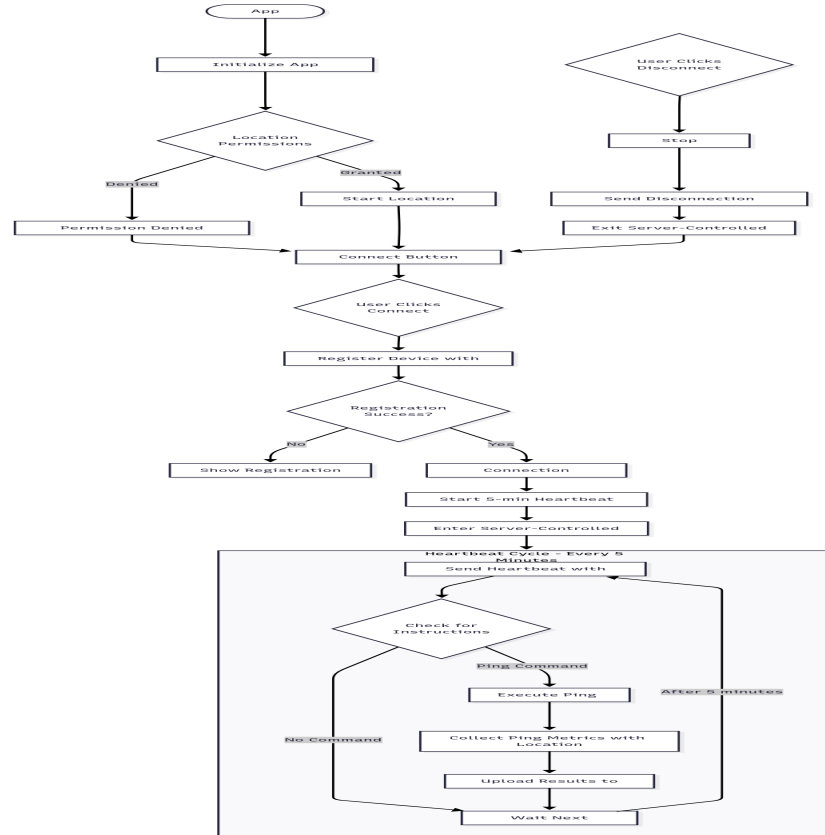


Figure 14: Enter Caption

9.1 Initialization

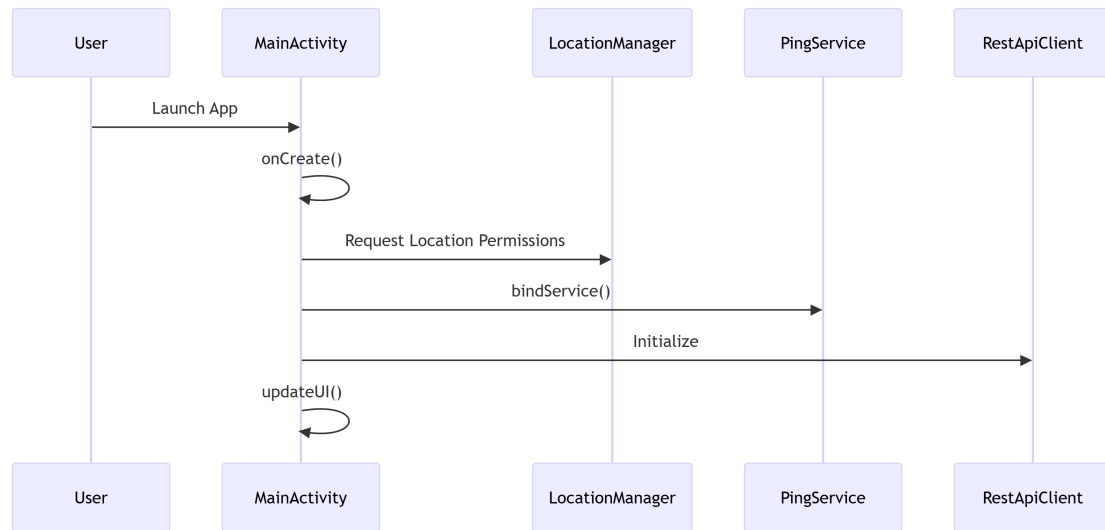


Figure 15:

1. **User launches the application** starting the **MainActivity** lifecycle.
2. **MainActivity – onCreate()** The `onCreate()` method of **MainActivity** is invoked.
 - UI components (buttons, text views, etc.) are bound.
 - Toolbar and menu are set up.
 - The **RestApiClient** is instantiated.
3. **Request Location Permissions** **MainActivity** checks for location access permissions.
 - If granted, location updates are initialized using the **LocationManager**.
4. **Bind to PingService** **MainActivity** binds to the background **PingService** using `bindService()`.
 - A connection is established between the activity and the service.
 - This allows the activity to call service methods and register callbacks.
5. **PingService Initialization** Once the service is bound:
 - **PingService** is initialized.
 - A notification channel is created for foreground execution.
 - Internal components (such as the **RestApiClient** inside the service) are prepared.
6. **Update UI State** Finally, **MainActivity** calls `updateUI()` to refresh the interface according to the current state of:
 - Service connection
 - Location availability
 - Connection status with the server

9.2 Server Connection

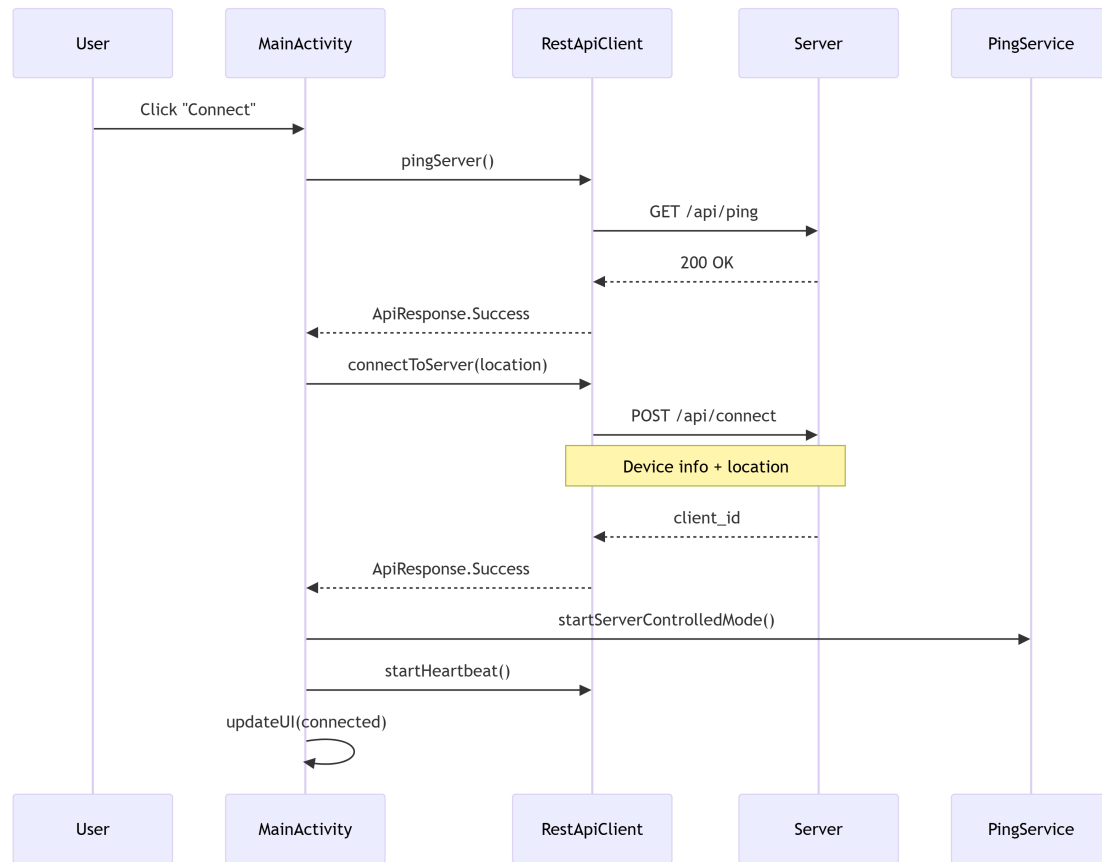


Figure 16:

1. **User Action: Click “Connect”**
2. **MainActivity – pingServer()** The activity first tests server availability by calling pingServer() on the RestApiClient.
 - This issues a GET /api/ping request to the server.
 - The server responds with a simple 200 OK and a message (pong).
3. **ApiResponse.Success (ping)** The RestApiClient notifies the MainActivity that the server is reachable by returning an ApiResponse.Success object.
4. **MainActivity – connectToServer(location)** Once server availability is confirmed, the activity proceeds to formally register the device
 - This issues a POST /api/connect request.
 - The request body contains device information such as:
 - Device ID (unique identifier)
 - Device model and Android version
 - Application version
 - Current device location
 - Timestamp of the request
5. **Server Response: client_id** The server stores or updates the device information in its database and generates a unique client_id. This client_id is returned to the application as confirmation of successful registration.
6. **ApiResponse.Success (connect)** The RestApiClient returns this server re-

sponse to the `MainActivity`, signalling that the connection was successful.

7. **Start Server-Controlled Mode** The `MainActivity` instructs the `PingService` to enter server-controlled mode. In this state, the service runs in the foreground and is ready to execute ping instructions received from the server.
8. **Start Heartbeat** The `MainActivity` also starts the heartbeat system via the `RestApiClient`.
 - This sends periodic heartbeat messages (default interval: 5 minutes).
 - Heartbeats include the client ID, device status, and location.
 - The server may respond with instructions for ping operations.
9. **Update UI (connected)**

9.3 Heartbeats

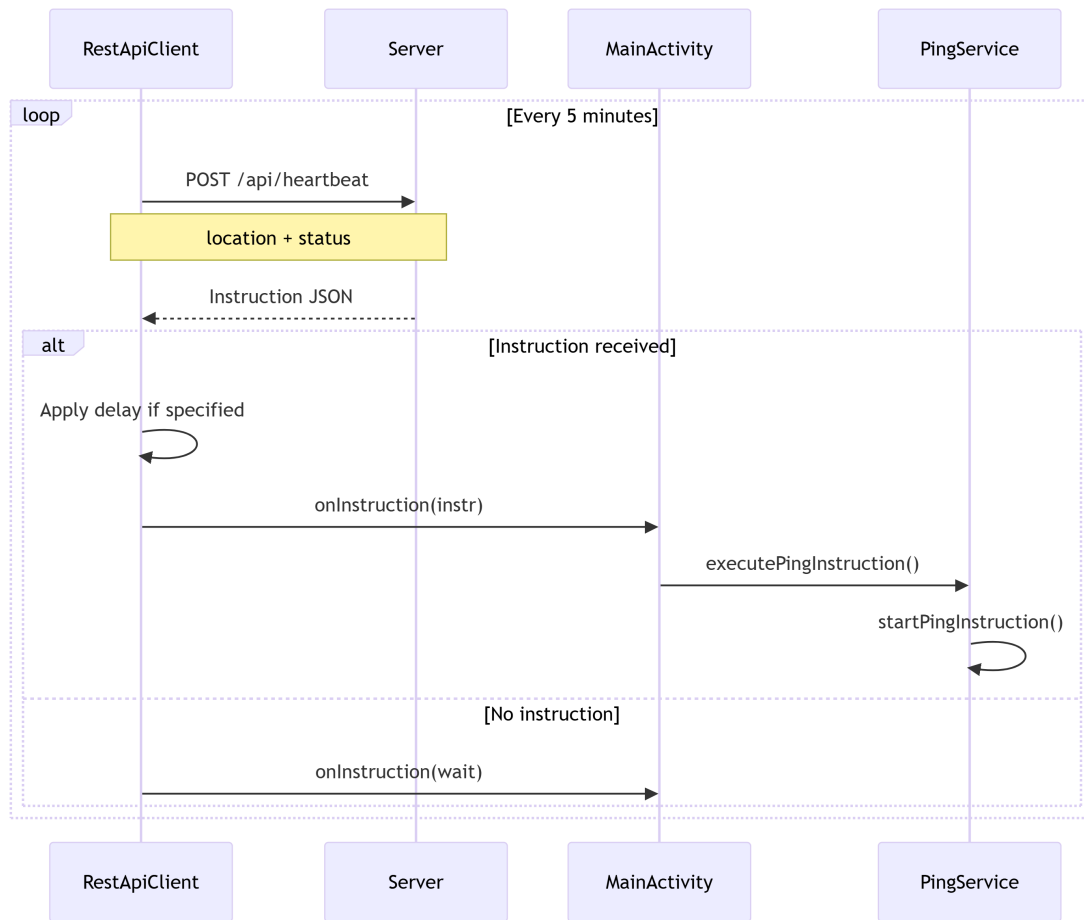


Figure 17:

1. **Heartbeat Timer (Every 5 Minutes)** The `RestApiClient` triggers a heartbeat every 5 minutes using an internal coroutine loop.
2. **`RestApiClient` – `sendHeartbeatAndCheckInstructions()`** The client sends a heartbeat request to check for server instructions.
 - Issues a `POST /api/heartbeat` request
 - Payload includes device ID, client ID, status, current location, and timestamp
3. **Server Processing** The server updates client data and decides whether to issue a ping instruction.
4. **Server Response with Instruction** When issuing an instruction, the server responds with ping parameters:
 - Target host, protocol (ICMP/TCP/UDP), duration, and interval
 - Unique instruction ID for tracking
 - Optional delay before execution
5. **Server Response without Instruction** When no instruction is issued, the server responds with acknowledgment only.
6. **`RestApiClient` – `Apply Delay`** If the server specifies a delay, the client waits before forwarding the instruction.
 - Extracts `delay_ms` from response
 - Applies `delay()` if value $\neq 0$

- Tracks delay time for heartbeat interval adjustment
7. **MainActivity – onInstruction() Callback** The client notifies the activity of the instruction via callback.
 8. **PingService – executePingInstruction()** The activity forwards the instruction to the background service.
 - Validates service is in server-controlled mode
 - Cancels existing operations if running
 - Initializes session tracking with unique session ID
 9. **Ping Execution Loop** The service executes pings according to the instruction parameters.
 10. **Heartbeat Timing Adjustment** The client adjusts next heartbeat timing to maintain 5-minute intervals despite instruction delays.

9.4 Pings

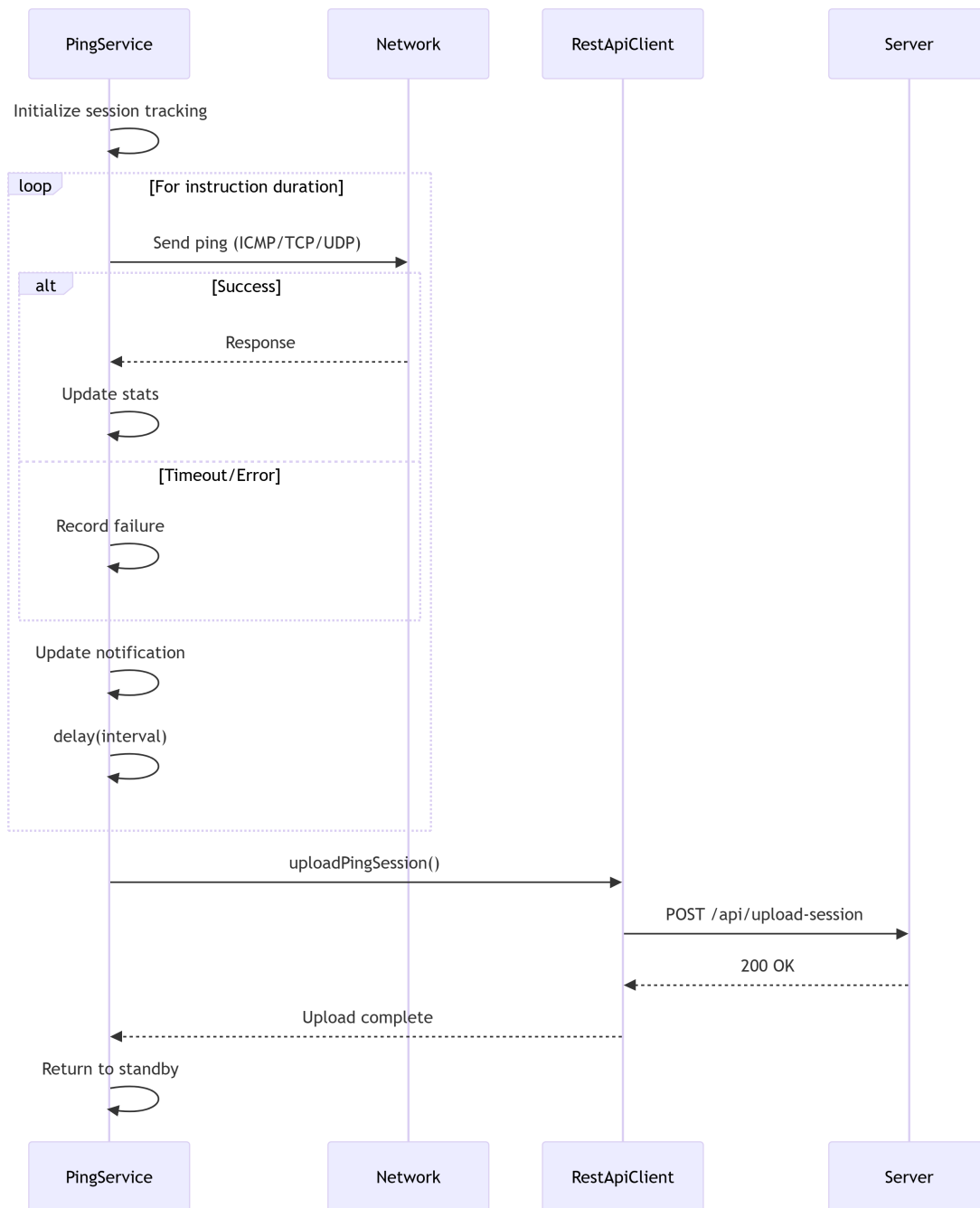


Figure 18:

1. **Send Ping Request** The service sends individual ping packets based on the specified protocol.
 - ICMP: Uses system `ping` command via `ProcessBuilder`
 - TCP: Creates socket connection to specified port
 - UDP: Sends datagram packet and waits for response
2. **Network Response (Success)** When the target host responds successfully:
 - Measures round-trip time (RTT) using `measureTimeMillis`
 - Increments packets received counter

- Updates RTT statistics (min, max, total for average calculation)
- 3. **Record Failure** The service logs the failed ping attempt for session reporting.
 - Stores failure in `pingResults` list with sequence number
 - Includes error message and timestamp for debugging
- 4. **Upload Session Data** Upon completion, the service calls `uploadPingSession()` via `RestApiClient`.
 - Creates `PingSessionData` object with session metrics
 - `RestApiClient` issues `POST /api/upload-session` request
 - Server stores data in `ping_sessions` and `ping_results` tables
- 5. **Server Confirmation** The server responds with upload confirmation.
- 6. **Upload Complete** The `RestApiClient` notifies the service of successful upload.
 - Logs successful upload in application log
 - Service can proceed to cleanup and standby state

1 September 2025

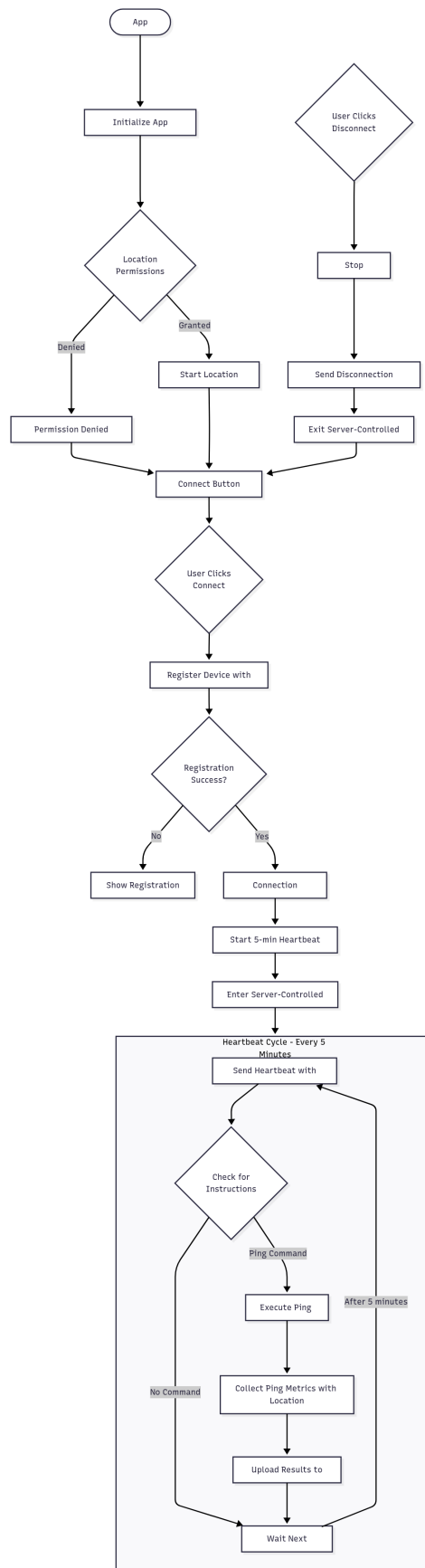


Figure 19: Enter Caption

2 November 2025

10 Summary

The provided code describes a distributed network monitoring system composed of an **Android application (client)** acting as a probe and a **Python-based backend (server)** functioning as a Command and Control (C&C) server.

The system's core purpose is to instruct Android devices to perform UDP packet tests from their unique network locations. The server collects detailed results, such as Round-Trip Time (RTT) and packet loss, to monitor network quality and reachability from a wide range of user perspectives.

The system features two distinct ping mechanisms:

- **App-to-Target Ping:** The server instructs the app to ping a *third-party target*. The app executes this ping and reports the results.
- **Server-to-App Ping:** The server itself pings the app (after the app signals it's ready) to measure connectivity and RTT *to* the device.

11 System Architecture

The system is split into two primary parts: the Android Client and the Backend Infrastructure.

11.1 Android Client (The "Probe")

The client is an Android application composed of a UI and two critical background services.

- **MainActivity.kt (UI Layer):**
 - Provides the user interface with "Connect" and "Disconnect" buttons.
 - Displays connection status and a scrolling log of activities.
 - Handles requesting all necessary permissions:
 - * **Location** (ACCESS_FINE_LOCATION, ACCESS_BACKGROUND_LOCATION)
 - * **Notifications** (POST_NOTIFICATIONS): Required for Android 13+ to run Foreground Services, which are essential for long-running background tasks.
- **ApiService.kt (Control Plane Service):**
 - A **Foreground Service** responsible for all communication with the `server.py` REST API.
 - Runs the persistent **heartbeat loop** (`/api/heartbeat`) to signal server availability and receive new instructions.
 - Handles the **Server-to-App ping** logic: it starts a UDP listener on a dynamic port and notifies `server_to_app_pinger.py` that it is "READY_FOR_PINGS".
 - Manages a retry queue for failed result uploads.
- **PingService.kt (Data Plane Service):**
 - A second **Foreground Service** responsible for *executing* the **App-to-Target** ping instructions received from `ApiService`.
 - Capable of performing UDP pings to any host specified by the server.
 - After a session, it hands the results back to `ApiService` for uploading.
- **NetworkUtils.kt (Utility):**

- A helper class to detect the current network type (e.g., "WiFi", "Cellular", "VPN").

11.2 Backend Infrastructure (The "Server")

The backend consists of three distinct Python servers.

- **server.py (Main C&C REST API):**
 - The "brain" of the operation; an **HTTPS REST API** server using `ssl.SSLContext`.
 - Listens on port 12345.
 - Uses **SQLite** (`ping_data.db`) for all data persistence.
 - **Endpoints:**
 - * `/api/connect`: Registers new clients.
 - * `/api/heartbeat`: Receives client status and always sends a new ping instruction back (with a randomized delay).
 - * `/api/upload-session`: Receives and stores the detailed results from the `PingService`.
 - * `/api/clients`: An admin endpoint to view registered clients.
- **server_to_app_pinger.py (Server-to-App Ping Server):**
 - A separate **UDP server** that listens on port 50003.
 - Its sole purpose is to receive "READY_FOR_PINGS" messages from the `ApiService`.
 - When it receives a "READY" message, it spawns a new thread and begins sending UDP "PING" messages to the client's dynamic port, listening for "ACK" messages to calculate RTT.
- **listner.py (UDP Echo Target):**
 - A simple UDP echo server listening on port 50002.
 - This is a test *target* for the **App-to-Target** pings, not part of the core C&C logic.

12 Core Functionality & Protocol Flow

This section describes the step-by-step lifecycle of the application.

12.1 Connection and Registration

1. **User Action:** The user opens `MainActivity` and taps "Connect".
2. **Client Request:** `ApiService.kt` sends an **HTTPS POST** request to `https://<server>:12345`
 - **Payload (JSON):** Contains `device_id`, `app-version`, `device_model`, `android-version`, and current location.
3. **Server Response:** `server.py` saves the client's info to the `clients` table in SQLite and responds with a `client_id`.
4. **Client State:** `ApiService` saves this `client_id`, enters a "Connected" state, and calls `startHeartbeat()`.

12.2 The Heartbeat and Instruction Loop

1. **Heartbeat:** `ApiService` starts a coroutine loop. Every 1 hour (as defined by `HEARTBEAT_INTERVAL_MS`), it sends an **HTTPS POST** to `/api/heartbeat`.
 - **Payload (JSON):** Contains `client_id`, `device_id`, `app-status`, and current location.

2. **Instruction:** `server.py` receives the heartbeat, logs it, and generates a new instruction using `get_ping_instruction()`. A randomized delay (averaging 30 minutes) is added to stagger the pings.
3. **Server Response (JSON):** The server sends the instruction back in the heartbeat response.

Listing 1: Example Heartbeat Response with Instruction

```
{
  "send_ping": true,
  "ping_host": "170.187.252.25",
  "ping_protocol": "UDP",
  "ping_duration_seconds": 15,
  "ping_interval_ms": 500,
  "delay_ms": 1800000
}
```

12.3 Ping Execution (Two Types Simultaneously)

When `ApiService` receives the instruction, it triggers two separate ping processes.

Type 1: App-to-Target Ping (via `PingService`)

1. **Delay:** `ApiService` first honors the `delay_ms` from the instruction.
2. **Delegation:** After the delay, `ApiService` signals `MainActivity`, which in turn calls `pingService.executePingInstruction()`.
3. **Execution:** `PingService` (as a Foreground Service) starts its ping loop, sending packets as instructed.
4. **Completion:** `PingService` stops, bundles all data into a `PingSessionData` object, and calls `apiService.uploadPingSession()`.
5. **Upload:** `ApiService` sends this data as an **HTTPS POST** to `/api/upload-session`.

Type 2: Server-to-App Ping (via `ApiService`)

1. **Simultaneous Action:** As soon as `ApiService` receives the instruction, it also calls `startUdpListener()`.
2. **Listener:** The app opens a UDP socket on a random, dynamic port (e.g., 45678).
3. **Notify Server:** The app sends 5 UDP packets to `server_to_app_pinger.py` at `<server_ip>:50003`.
4. **Server Action:** `server_to_app_pinger.py` receives this "READY" message and starts a 30-second ping session *to* the client's address.
5. **Client ACK:** `ApiService`'s listener receives this and sends back
6. **Timeout:** The app's listener closes after 60 seconds.

13 Data and Protocols Summary

- **HTTPS (Port 12345):** Used for the **Control Plane**.
 - **Protocol:** Encrypted REST API (HTTP over TLS/SSL).
 - **Data:** JSON payloads for registration, heartbeats, instructions, and result uploads.

- **Security:** `server.py` uses `cert.pem` and `key.pem` files to provide encryption.
- **UDP (Various Ports):** Used for the **Data Plane** (the actual pings).
 - **App-to-Target (e.g., Port 50002):** `PingService` sends raw UDP packets (or ICMP/TCP) to a target and waits for a response. This measures *external* network quality.
 - **Server-to-App (Ports 50003 & Dynamic):** `ApiService` listens on a dynamic port. `server_to_app_pinger.py` sends JSON-formatted UDP packets to it. This measures *client reachability*.

14 Key File Analysis

- **MainActivity.kt:** The entry point and UI controller. Its main jobs are handling permissions and binding to the `ApiService` and `PingService` to orchestrate the UI with the background logic.
- **PingService.kt:** The "worker" service. Executes the **App-to-Target** pings. It's well-structured to handle ICMP, TCP, and UDP pings, track detailed results, and manage session state.
- **ApiService.kt:** The "manager" service. This is the most complex client-side component. It manages the app's entire lifecycle state, handles all C&C communication, and implements the listener for the **Server-to-App** ping.
- **NetworkUtils.kt:** A standard Android utility for getting network-aware information, which is used to enrich the ping results.
- **server.py:** The C&C server. A standard Python `HTTPServer` with a RESTful API. It's the source of truth, issuing all commands and storing all results in its SQLite database.
- **server_to_app_pinger.py:** A specialized UDP server for the **Server-to-App** ping. This design choice separates the stateful UDP socket operations from the stateless REST API server.
- **listner.py:** A basic UDP echo server. This is a *tool*,